

Technical Architecture and Research for a Unity 2026 Split-Screen Cooperative Dungeon Crawler

Executive Summary and Game Design Document

The development of a robust, local split-screen multiplayer application within the Unity 2026 engine demands a highly modular, scalable, and performant architectural foundation. To satisfy the rigorous technical constraints outlined by the Graphics Unity Project 2026¹, the software architecture must seamlessly integrate the Unity New Input System for simultaneous local device polling, utilize ScriptableObject data containers for memory-efficient state management, and employ procedural generation algorithms to synthesize dynamic environments.² The integration of these systems ensures that the application not only meets the fundamental requirements of a shared-screen cooperative experience but also scales efficiently without incurring technical debt.

The following Game Design Document (GDD) outlines the core parameters, thematic setting, and entity classifications required to successfully implement the "Fantasy Dungeon" theme while fulfilling all academic project criteria, including basic requirements, extra functionalities, and potential bonus implementations.¹

Thematic Setting and Core Objectives

The application, internally designated as *Echoes of the Obsidian Citadel*, functions as a cooperative (Co-op) 3D dungeon crawler where two users must navigate procedurally generated subterranean labyrinths. Set in a dark fantasy epoch, players assume the roles of specialized adventurers attempting to breach the deeper sanctums of a ruined fortress. The primary objective requires players to collaboratively survive hostile encounters, gather necessary consumables, circumvent physical obstacles, and defeat the sanctum's artificial intelligence (AI) guardians to unlock the exit to the subsequent procedural tier.¹

The game loop is inherently reliant on interdependent mechanics. Neither player possesses the complete toolkit required to survive the procedural generation alone; they must rely on the asymmetrical class design to overcome obstacles.

Entity Classification and Mechanics

The application of a modular design philosophy necessitates the strict categorization of in-game entities. The architecture relies on an overarching taxonomy to define Playable Characters (PCs), Non-Playable Characters (NPCs), and environmental interactables.¹

To fulfill the constraint that each player must be able to choose between two distinct characters, the system partitions the four classes into two distinct archetypal pools. Player 1 selects from high-damage/agile classes, while Player 2 selects from high-durability/support classes. The table below delineates the unique mechanical identifiers of these classes.

Playable Character Class	Selection Pool	Primary Role	Movement/Combat Style	Unique Mechanical Identifier
Wizard	Player 1	Area of Effect (AOE) / Damage	Ranged / Low Agility	Casts projectile-based physics triggers that instantiate radial damage zones upon collision.
Assassin	Player 1	Burst Damage / Skirmisher	Melee / High Agility	Utilizes transform-based dash mechanics and elevated base locomotion velocity to bypass frontline enemies.
Fighter	Player 2	Tank / Crowd Control	Melee / Low Agility	Employs expanded health pools and physical mass to block chokepoints, utilizing Physics.OverlapSphere for melee sweeps.
Cleric	Player 2	Support / Sustenance	Ranged / Medium Agility	Generates healing auras via localized triggers

				that restore hit points to allied entities within a specific mathematical radius.
--	--	--	--	---

To challenge these players, the application deploys hostile NPCs governed by a Finite State Machine (FSM) architecture. These entities utilize dynamic pathfinding to navigate the procedurally generated terrain, recalculating their vectors in real-time as physical obstacles are instantiated or destroyed.

Non-Playable Character (NPC)	AI Behavior Profile	NavMesh Interaction	Encounter Role
Undead Sentinel (Hostile)	FSM (Patrol, Chase, Attack)	Dynamic Pathfinding	Basic melee adversary that pursues the nearest player transform upon entering a predefined scalar detection radius.
Spectral Mage (Hostile)	FSM (Idle, Flee, Attack)	Dynamic Pathfinding	Maintains geometric distance from players; fires linear physics projectiles when line-of-sight raycasts are successfully established.

The environment itself is populated with procedurally scattered interactables. These entities are segregated into consumables and rigid obstacles, each bearing specific Unity physics components to facilitate interaction.

Environment & Items	Interaction Type	Physics/Component Profile	Gameplay Function
Health Potion	Instant Consumable	SphereCollider (IsTrigger = true)	Restores a fixed integer of health upon intersection with a player's physics boundary, immediately destroying itself. ¹
Speed Booster	Inventory Consumable	BoxCollider (IsTrigger = true)	Appended to the player's ScriptableObject inventory list; grants a temporary velocity multiplier when explicitly activated via input. ¹
Fallen Pillar	Physical Obstacle	MeshCollider, NavMeshObstacle	Blocks player translation and forces the AI to recalculate paths by dynamically carving boolean holes into the NavMesh surface. ⁵

Game Logic Flowchart

To ensure a continuous gameplay loop that fulfills the basic mechanics and game manager requirements ¹, the system's operational logic is defined hierarchically. This text-based logic flow dictates the sequence of events from engine initialization to the resolution of terminal game states. The flow is designed to be deterministic, ensuring that edge cases—such as simultaneous player death or hardware disconnection—are handled gracefully. The following sequence details the application lifecycle state transitions.

System State	Execution Logic	Transition Condition

1. Initialization	Load MainMenuScene. Initialize UI Canvas. Render Start Game, Tutorial, Credits, and Quit buttons.	User selects "Start Game" → Transition to Character Selection. User selects "Quit" → Terminate Application.
2. Character Selection	Render selection UI. Poll for Player 1 input (Wizard/Assassin) and Player 2 input (Fighter/Cleric). Store selections in a persistent data container.	Both users confirm selection → Transition to GameplayScene.
3. Spawning & Pairing	PlayerInputManager listens for device connections. Player 1 (Keyboard/Mouse) instantiates PC 1. Player 2 (Gamepad or Keyboard Fallback) instantiates PC 2.	Both prefabs instantiated and Viewport Rects assigned → Proceed to Procedural Generation.
4. Procedural Generation	Execute Spatial Sampling Algorithm. Place bounds. Scatter obstacles and hostile NPCs. Bake/Update NavMeshSurface.	Generation loop concludes without spatial overlap errors → Proceed to Core Loop.
5. Core Gameplay Loop	Execute per-frame updates. Read continuous inputs (Movement, Camera). Resolve physics forces. Iterate NPC FSMs. Synchronize UI HUDs.	Frame rendered → Evaluate Win/Loss Conditions.

6. Condition Resolution	Evaluate player health integers and positional data against the exit volume trigger.	If Player 1 Health ≤ 0 AND Player 2 Health $\leq 0 \rightarrow$ Trigger Game Over UI $\rightarrow$ Return to State 1. If players collide with Level Exit $\rightarrow$ Increment Level Index $\rightarrow$ Return to State 4.
--------------------------------	--	--

Modular Architecture: Scriptable Objects and Class Management

A primary technical challenge in developing an application featuring four highly distinct playable classes is mitigating code duplication and avoiding monolithic, heavily branched conditional logic. Traditional approaches involving deeply nested switch statements or massive inheritance trees attached to MonoBehaviour scripts often lead to severe memory fragmentation and unmaintainable codebases.⁶

To achieve a highly modular and performant architecture, the system relies on the ScriptableObject pattern. ScriptableObjects are serializable data containers that exist at the project asset level, functioning entirely independently of scene instantiations.² By utilizing these objects, developers can isolate gameplay data—such as maximum health, locomotion speed multipliers, and attack damage integers—from behavioral execution logic.⁸ Furthermore, applying inheritance directly to ScriptableObjects permits the creation of a plug-and-play ability system where a unified PlayerController and CombatSystem can govern all four classes simultaneously.⁹

Base Class Data Structures

The foundation of the modular class system is an abstract data container that defines the universal parameters shared by the Wizard, Assassin, Fighter, and Cleric. This base class forces derived objects to implement their own specific combat execution methods, guaranteeing polymorphism.

```

C#
using UnityEngine

// Defines the base template for any playable class in the game
public abstract class CharacterClassSO : ScriptableObject
{

    public string className
    public int maxHealth
    public float baseMovementSpeed
    public float attackCooldown

    Header("Visual Assets")
    public GameObject characterModelPrefab
    public Sprite classIcon

    // Abstract method forcing specific classes to define their unique combat behavior
    public abstract void ExecuteAction(Transform playerTransform, Animator animator)
}

```

By defining ExecuteAction as an abstract method, the architecture mandates that specialized assets provide their own unique implementation. The data and the logic to execute that data are encapsulated entirely within the asset file. The following boilerplate code demonstrates how the Wizard and Fighter classes diverge in their mechanical execution.

```

C#
using UnityEngine

public class WizardClassSO : CharacterClassSO
{

    public GameObject magicProjectilePrefab
    public float projectileSpeed

    public override void ExecuteAction(Transform playerTransform, Animator animator)
    {
        animator.SetTrigger("CastSpell")
    }
}

```

```

// Instantiate a physics-based projectile for ranged AOE damage
Vector3 spawnPos = playerTransform.position + (playerTransform.forward * 1.5f);
Vector3 up =
GameObject projectile = Instantiate(magicProjectilePrefab, spawnPos,
playerTransform.rotation);

```

```

Rigidbody rb = projectile.GetComponent<Rigidbody>();
if (rb != null)
{
rb.AddForce(playerTransform.forward * projectileSpeed, ForceMode.Impulse);
}
}

```

```

CreateAssetMenu(fileName = "NewFighterClass", menuName = "Classes/Fighter")
public class FighterClassSO : CharacterClassSO
{

```

```

public float meleeRange;
public int meleeDamage;
public LayerMask enemyLayer;

```

```

public override void ExecuteAction(Transform playerTransform, Animator animator)
{
animator.SetTrigger("MeleeSwing");
}

```

```

// Implementation of Physics.OverlapSphere to detect enemies in melee range
Vector3 strikeCenter = playerTransform.position + (playerTransform.forward * 1.0f);
Collider[] hitEnemies = Physics.OverlapSphere(strikeCenter, meleeRange, enemyLayer);

```

```

foreach (Collider enemy in hitEnemies)
{
EnemyAI aiComponent = enemy.GetComponent<EnemyAI>();
if (aiComponent != null)
{
aiComponent.TakeDamage(meleeDamage);
}
}
}

```


The Unified Player Controller and Combat System

With the data and unique combat logic encapsulated within the asset files, the components attached to the player's root GameObject remain incredibly lightweight. To fulfill the specific architectural mandate of separating movement logic from offensive logic, the design utilizes two distinct MonoBehaviour scripts: the PlayerController and the CombatSystem.

The PlayerController acts exclusively as a physical vessel. It parses spatial inputs and translates the CharacterController component based on the speed variables dictated by the assigned ScriptableObject.

```
C#
using UnityEngine;
using UnityEngine.InputSystem;

public class PlayerController : MonoBehaviour
{
    public CharacterClassSO currentClass;

    private CharacterController charController;
    public Animator animator { get; private set; }

    private Vector2 movementInput;
    public int currentHealth { get; set; }

    void Awake()
    {
        charController = GetComponent<CharacterController>();
    }

    public void InitializeClass(CharacterClassSO selectedClass)
    {
        currentClass = selectedClass;
        currentHealth = currentClass.maxHealth;

        // Instantiate the specific 3D model assigned to this class
        GameObject model = Instantiate(currentClass.characterModelPrefab, transform);
    }
}
```

```

        animator = model.GetComponent<Animator>()
    }

    public void OnMove(InputValue value)
    {
        movementInput = value.Get<Vector2>()

    }

    void Update()
    {
        if (currentClass == null) return

        Vector3 move = new Vector3(movementInput.x, 0, movementInput.y)

        // Apply gravity and locomotion translation
        charController.Move(move * currentClass.baseMovementSpeed + Physics.gravity) *
        Time.deltaTime)

        if (move != Vector3.zero)
        {
            gameObject.transform.forward = move
            animator.SetBool("IsWalking", true)
        }
        else
        {
            animator.SetBool("IsWalking", false)
        }
    }
}

```

Conversely, the CombatSystem subscribes to the action inputs generated by the Unity New Input System. When a player triggers the attack command, the CombatSystem verifies cooldown thresholds and then blindly delegates the execution back to the ScriptableObject.

```

C#
using UnityEngine
using UnityEngine.InputSystem

```

```

public class CombatSystem : MonoBehaviour
{
    private PlayerController playerController;
    private float lastAttackTime;

    void Awake()
    {
        playerController = GetComponent<PlayerController>();
    }

    public void OnAttack(InputValue value)
    {
        if (playerController.currentClass == null) return;

        if (value.isPressed && Time.time >= lastAttackTime +
            playerController.currentClass.attackCooldown)
        {
            // The combat system does not know what the attack is; it delegates to the SO
            playerController.currentClass.ExecuteAction(this.transform,
                playerController animator);
            lastAttackTime = Time.time;
        }
    }
}

```

This structural paradigm provides immense scalability. Should the project requirements expand in the future to include a fifth class, developers merely execute a context menu command in the Unity Editor to create a new CharacterClassSO asset, populate the exposed variables, and inject it into the selection pool. Absolute zero alterations to the core PlayerController or CombatSystem scripts are required to facilitate this expansion.⁶

Input Subsystem and Local Multiplayer Fallback Topologies

The requirement to support two local players utilizing different control paradigms natively necessitates the deployment of the Unity New Input System package. The legacy input manager is fundamentally incapable of gracefully routing simultaneous discrete device inputs—particularly multiple gamepads or keyboard splits—without extensive and fragile hardcoding.³

To fulfill the specific criteria—where Player 1 utilizes a Keyboard and Mouse (KBM), Player 2 utilizes a Gamepad, and a fallback topology is supported where both players utilize a single keyboard—the architecture leverages the PlayerInputManager component set to the Join

Players When Join Action Is Triggered behavior.³

Control Scheme Definitions and Action Maps

Within the global Input Action Asset, the architecture demands the authoring of distinct Control Schemes to prevent input bleeding. The bindings must be explicitly mapped as follows:

Control Scheme	Movement Binding	Action Binding	Device Requirement
KeyboardLeft	W, A, S, D	'F' Key	Keyboard
KeyboardRight	Up, Down, Left, Right	'Right Control' Key	Keyboard
Gamepad	Left Stick	South Button (A/Cross)	Gamepad

When the application initializes the GameplayScene, the PlayerInputManager awaits the designated "Join" button press from any connected device. The critical challenge lies in handling the keyboard fallback condition if a physical gamepad is entirely absent from the host machine.¹¹ The system must explicitly permit the keyboard to be logically partitioned into multiple virtual devices.

Dynamic Device Pairing and Fallback Architecture

To manage this complex routing, a MultiplayerGameManager script subscribes to the C# events invoked by the PlayerInputManager. When a player joins, the manager evaluates the chronological join order and the hardware type of the activating device.

```
C#
using UnityEngine;
using UnityEngine.InputSystem;
```

```

public class MultiplayerGameManager : MonoBehaviour
{
    public PlayerInputManager inputManager;
    public Transform spawnPoints;

    // Arrays representing the pre-selected classes from the menu
    public CharacterClassSO availableClassesP1;
    public CharacterClassSO availableClassesP2;

    private int playersJoined = 0;

    void OnEnable()
    {
        inputManager.onPlayerJoined += HandlePlayerJoin;
    }

    void OnDisable()
    {
        inputManager.onPlayerJoined -= HandlePlayerJoin;
    }

    private void HandlePlayerJoin(PlayerInput playerInput)
    {
        if (playersJoined >= 2) return;

        // Route the newly spawned player to the correct physical location
        playerInput.transform.position = spawnPoints[playersJoined].position;

        PlayerController pController = playerInput.GetComponent<PlayerController>();

        // Hardware Fallback Logic: Detect if the input stems from a keyboard
        bool isKeyboard = playerInput.devices.Count > 0 && playerInput.devices[0] is Keyboard;

        if (playersJoined == 0)
        {
            // Player 1 Initialization
            pController.InitializeClass(availableClassesP1); // Default to first choice (Wizard)
        }

        if (isKeyboard)
        {
            playerInput.SwitchCurrentControlScheme("KeyboardLeft", Keyboard.current);
        }
    }
}

```

```

else if (playersJoined == 1)
{
    // Player 2 Initialization
    pController.InitializeClass(availableClassesP2) // Default to first choice (Fighter)

    if (!isKeyboard)
    {
        // Force the right-side keyboard mapping to prevent P1 interference
        playerInput.SwitchCurrentControlScheme("KeyboardRight", Keyboard.current);
    }

    // If the device is a Gamepad, the Input System natively resolves it via the Gamepad scheme
}

playersJoined++;
}

```

This robust initialization guarantees that evaluators grading the academic project can test both players seamlessly on a single laptop without requiring peripheral hardware, directly satisfying the project constraints regarding input fallback mechanisms.¹

Split-Screen Dual Camera Engineering and Viewport Rects

To render the three-dimensional environment for two separate users simultaneously, the rendering pipeline must geometrically divide the screen real estate. This division is achieved by mathematically manipulating the Viewport Rect properties of the Unity Camera components attached to each player prefab.¹³

The Viewport Rect utilizes normalized screen coordinates ranging from 0.0 to 1.0, rather than absolute pixel values. This ensures that the split-screen paradigm remains resolution-independent, functioning identically on a 1080p monitor as it does on a 4K display. A standard horizontal split-screen configuration divides the screen vertically precisely down the middle.¹³

Camera Assignment	Rect X (Horizontal Offset)	Rect Y (Vertical Offset)	Rect Width	Rect Height
Player 1 Camera	0.0	0.0	0.5	1.0

Player 2 Camera	0.5	0.0	0.5	1.0
--------------------	-----	-----	-----	-----

Because the PlayerInputManager instantiates identical prefabs from the project directory, the camera components within those prefabs will initially overlap, rendering at a default width of 1.0. The architecture must automatically calculate and adjust the rects based on the player's chronological index upon instantiation.¹²

Viewport Rect Boilerplate Logic

The following component is attached to the root of the player prefab. Upon instantiation, it identifies its owning player index and alters the rendering pipeline accordingly.

```
C#
using UnityEngine;
using UnityEngine.InputSystem;

public class SplitScreenCameraSetup : MonoBehaviour

{
    public Camera playerCamera;

    // Variables to support independent POV switching
    public Vector3 defaultCameraOffset = new Vector3(0, 5, -8);
    public Vector3 topDownCameraOffset = new Vector3(0, 15, 0);
    private bool isTopDown = false;

    void Start()
    {
        PlayerInput plnput = GetComponent<PlayerInput>();
        int playerIndex = plnput.playerIndex;

        // Dynamic viewport allocation based on join order
        if (playerIndex == 0)
```

```

        playerCamera.rect = new Rect(0f, 0f, 0.5f, 1f) // Left Hemisphere
    }
    else if (playerIndex == 1)
    {
        playerCamera.rect = new Rect(0.5f, 0f, 0.5f, 1f) // Right Hemisphere

        // Resolve AudioListener conflict: Unity only permits one active listener per scene
        AudioListener listener = playerCamera.GetComponent<AudioListener>();
        if (listener != null) Destroy(listener);
    }

    // Fulfills the requirement for independent POV switching
    public void OnToggleCamera(InputValue value)
    {
        if (value.isPressed)
        {
            isTopDown = !isTopDown;
            Transform camTransform = playerCamera.transform;

            if (isTopDown)
            {
                camTransform.localPosition = topDownCameraOffset;
                camTransform.localRotation = Quaternion.Euler(90, 0, 0);
            }
            else
            {
                camTransform.localPosition = defaultCameraOffset;
                camTransform.localRotation = Quaternion.Euler(30, 0, 0);
            }
        }
    }
}

```

This methodology provides entirely independent Points of View (POV) that are segregated from one another. Should Player 1 wish to toggle their perspective to a top-down view to navigate a complex puzzle, the script manipulates the camera's local transform and rotation without exerting any influence over Player 2's viewport, fulfilling the advanced camera UI requirements mandated by the project.¹

Lightweight Inventory and Consumables Architecture

The project specification mandates the implementation of an inventory subsystem capable of handling various consumables (e.g., Health Potions, Speed Boosters). These items must trigger

their respective gameplay effects either instantaneously upon environmental collision or via discrete button presses from a stored inventory state.¹

A traditional, heavily coupled inventory array—where strings or enumerations dictate item logic—is prone to referencing errors and necessitates constant modification of the inventory script whenever a new item is introduced. Applying the ScriptableObject methodology once again yields a highly optimized, lightweight solution. The data container defines the item's identity, while inheritance and polymorphism dictate its specific effect on the game world.¹⁰

Consumable Data Structures

```
C#
using UnityEngine;

public abstract class ConsumableSO : ScriptableObject
{
    Header("Item Metadata")
    public string itemName;
    public Sprite inventoryIcon;

    public bool activateOnPickup;

    // Passes the PlayerController to mutate the specific player's state
    public abstract void UseItem(PlayerController player);
}

public class SpeedBoosterSO : ConsumableSO
{
    public float speedMultiplier = 1.5f;

    public override void UseItem(PlayerController player)
    {
        // Temporarily mutates the movement speed logic
        player.currentClass.baseMovementSpeed *= speedMultiplier;
        Debug.Log($"{player.name} used {itemName} and increased speed.");
        // A coroutine would subsequently revert this value after X seconds
    }
}
```

The World-Space Trigger and Inventory Manager

In the 3D scene, a physical representation of the consumable contains a SphereCollider with the `IsTrigger` parameter enabled. When a player's physics boundary overlaps this trigger, the `InventoryManager` intercepts the collision. The manager evaluates the `activateOnPickup` boolean to determine whether the item instantly resolves or appends itself to the player's local inventory.¹

```
C#
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.InputSystem;

public class InventoryManager : MonoBehaviour
{
    private PlayerController playerController;

    // The lightweight inventory simply stores memory references to the ScriptableObjects
    public List<ConsumableSO> storedItems = new List<ConsumableSO>();

    void Awake()
    {
        playerController = GetComponent<PlayerController>();
    }

    // Handles the physics collision with objects in the environment
    private void OnTriggerEnter(Collider other)
    {
        if (other.CompareTag("Consumable"))
        {
            PickupItem pickup = other.GetComponent<PickupItem>();
            if (pickup != null && pickup.itemData != null)
            {
                if (pickup.itemData.activateOnPickup)
```

```

        // Instant environmental effect (e.g., ground speed pad)
        pickup.itemData.UseItem(playerController);

    else
        // Store the memory reference for later use
        storedItems.Add(pickup.itemData);
        UpdateInventoryUI();

    // Destroy the physical mesh in the world
    Destroy(other.gameObject);

    // Handles the discrete input command to consume a stored item
    public void OnUseItem(InputValue value)
    {
        if (value.IsPressed && storedItems.Count > 0)
        {
            // Retrieve and execute the most recently picked up item
            ConsumableSO itemToUse = storedItems[storedItems.Count - 1];
            itemToUse.UseItem(playerController);

            // Remove from the data structure
            storedItems.RemoveAt(storedItems.Count - 1);
            UpdateInventoryUI();
        }
    }

    private void UpdateInventoryUI()
    {
        // Logic to dispatch a C# event to the Canvas HUD to update Sprite graphics
    }
}

```

This decoupled system ensures that new power-ups and consumables can be integrated purely through the Unity Editor's asset creation menu without writing new conditional logic inside the interaction loop, directly satisfying the requirement for modular mechanics.

Artificial Intelligence: Finite State Machines and

NavMesh Navigation

Non-Playable Characters (NPCs) must exhibit autonomous behaviors utilizing a Finite State Machine (FSM) and dynamic pathfinding to fulfill the artificial intelligence requirements.¹ Unity's built-in NavMeshAgent handles spatial translation, avoiding physical obstacles and calculating optimal routes, but the cognitive decision-making logic governing *when* and *where* to move must be explicitly programmed.

The FSM transitions between distinct behavioral phases based on environmental variables, specifically evaluating the geometric distance between the NPC's transform and the nearest player transform.

Behavioral State	Execution Logic	Conditional Transition Trigger
Idle	NavMeshAgent velocity forced to 0. Animator transitions to a breathing or waiting loop.	Internal timer threshold exceeded → Switch to Patrol.
Patrol	Agent selects a random vector coordinate within a predefined scalar radius and translates toward it.	Distance to Player < Detection Radius → Switch to Chase.
Chase	Agent queries the NavMesh for the shortest contiguous path to the Player and updates its destination dynamically.	Distance to Player < Attack Radius → Switch to Attack. Distance > Detection Radius → Revert to Patrol.
Attack	Agent ceases movement. Triggers combat animation sequence and executes damage calculation against	Distance to Player > Attack Radius → Revert to Chase.

	the player.	
--	-------------	--

FSM Boilerplate Architecture

The following class demonstrates how the EnemyAI script manages these state transitions sequentially within the Update loop.

```
C#
using UnityEngine;
using UnityEngine.AI;

public class EnemyAI : MonoBehaviour
{
    public enum State { Idle, Patrol, Chase, Attack }
    public State currentState = State.Idle;

    private NavMeshAgent agent;
    private Transform targetPlayer;
    private Animator animator;

    Header("AI Parameters")
    public float detectionRadius = 10f;
    public float attackRadius = 2f;
    public int attackDamage = 15;

    private float stateTimer = 0f;

    void Start()
    {
        agent = GetComponent<NavMeshAgent>();
        animator = GetComponent<Animator>();
    }

    void Update()
```

```

    switch (currentState)
    {
        case State.Idle:
            animator.SetBool("IsMoving" false);
            stateTimer += Time.deltaTime;
            if (stateTimer > 2f)
            {
                stateTimer = 0f;
                currentState = State.Patrol;
                CheckForPlayer();
                break;
            }

        case State.Patrol:
            animator.SetBool("IsMoving" true);
            if (agent.pathPending && agent.remainingDistance < 0.5f)
            {
                // Generate a random Cartesian coordinate for the patrol route
                Vector3 randomDir = Random.insideUnitSphere * 5f;
                randomDir += transform.position;

                NavMeshHit hit;
                if (NavMesh.SamplePosition(randomDir, out hit, 5f, 1))
                {
                    agent.SetDestination(hit.position);
                    CheckForPlayer();
                    break;
                }

            }

        case State.Chase:
            if (targetPlayer != null)
            {
                animator.SetBool("IsMoving" true);
                agent.SetDestination(targetPlayer.position);

                if (Vector3.Distance(transform.position, targetPlayer.position) <= attackRadius)
                {
                    currentState = State.Attack;
                }
                else if (Vector3.Distance(transform.position, targetPlayer.position) <
                    detectionRadius)
            }

```

```

        }

        targetPlayer = null
        currentState = State.Patrol

    }

    break

    case State.Attack:
        agent.SetDestination(transform.position); // Halt agent translation
        animator.SetBool("IsMoving" false)
        transform.LookAt(new Vector3(targetPlayer.position.x, transform.position.y,
targetPlayer.position.z));

        stateTimer += Time.deltaTime;
        if (stateTimer > 1.5f) // Time between strikes
        {
            animator.SetTrigger("MeleeSwing")
            targetPlayer.GetComponent<PlayerController>().currentHealth -=
attackDamage;
            stateTimer = 0f;
        }

        if (Vector3.Distance(transform.position, targetPlayer.position) > attackRadius)
        {
            currentState = State.Chase;
        }

        break;
    }

    private void CheckForPlayer()
    {
        // Query the physics engine for all potential targets
        Collider hits = Physics.OverlapSphere(transform.position, detectionRadius);
        float closestDistance = Mathf.Infinity;

        foreach (Collider hit in hits)
        {
            if (hit.CompareTag("Player"))
            {
                float dist = Vector3.Distance(transform.position, hit.transform.position);
                if (dist < closestDistance)
                {

```

```

        closestDistance = distance;
        targetPlayer = hit.transform.GetComponent<Player>();
        currentState = State.Chase;
    }

    void Update()
    {
        // Update logic
    }
}

```

Procedural Map Generation and Spatial Sorting

To secure the extra 20% grading requirement for Procedural Map Generation¹, the environment cannot rely entirely on static, hand-placed prefabs. The algorithm must procedurally scatter physical obstacles, interactive consumables, and hostile NPCs across the playing field. Crucially, it must guarantee mathematically that entities do not spawn overlapping each other or clipping through level boundaries.¹⁸

The Spatial Sampling Algorithm

While highly advanced techniques like Poisson Disc Sampling offer mathematically perfect distributions and tight packing¹⁹, a highly effective and far less computationally expensive heuristic for this specific academic scope involves randomized coordinate generation validated through Physics Application Programming Interfaces (APIs). Utilizing `Physics.CheckSphere` creates a lightweight placement algorithm that queries the spatial volume before approving an instantiation command.²⁰

```

C#
using UnityEngine;

public class ProceduralGenerator : MonoBehaviour
{
    // Prefabs
    public GameObject obstaclePrefab;
    public GameObject npcPrefab;
    public GameObject consumablePrefab;

    // Configuration
    public int numberOfObstacles = 20;
    public int numberOfNPCs = 5;
}

```



```
public int numberOfConsumables = 10
```

```
public Vector2 mapSize = new Vector2(40f, 40f)
```

```
public float overlapCheckRadius = 1.5f
```

```
public LayerMask collisionMask
```

```
void Start()
```

```
    GenerateObjects(obstaclePrefabs, numberOfObstacles)
```

```
    GenerateObjects(npcPrefabs, numberOfNPCs)
```

```
    GenerateObjects(consumablePrefabs, numberOfConsumables)
```

```
void GenerateObjects(GameObject prefabArray, int targetCount)
```

```
    int objectsSpawned = 0
```

```
    int attempts = 0 // Safety breakout to prevent infinite while-loops
```

```
    while (objectsSpawned < targetCount && attempts < 1000)
```

```
        attempts++
```

```
        // Generate a random Cartesian coordinate within the specified floor bounds
```

```
        Vector3 randomPos = new Vector3
```

```
            Random.Range(-mapSize.x / 2, mapSize.x / 2)
```

```
            0.5f // Offset slightly above the floor plane
```

```
            Random.Range(-mapSize.y / 2, mapSize.y / 2)
```

```
        // Query the physics engine: Is this spherical volume empty?
```

```
        if (!Physics.CheckSphere(randomPos, overlapCheckRadius, collisionMask))
```

```
            // Select a random prefab from the provided array parameter
```

```
            GameObject selectedPrefab = prefabArray
```

```
            // Randomize the Y-axis rotation for visual variance
```

```
            Quaternion randomRot = Quaternion.Euler(0, Random.Range(0, 360), 0)
```

```
            Instantiate(selectedPrefab, randomPos, randomRot, this.transform)
```

```
            objectsSpawned++
```



NavMesh Integration for Procedural Geometry

A critical complication introduced by procedural generation is its catastrophic impact on pre-baked Artificial Intelligence navigation data. If a massive boulder is spawned randomly in the center of a corridor, the statically baked NavMesh will not recognize the new collision data. Consequently, NPCs will attempt to walk directly through the object, breaking immersion and violating physical constraints.

To rectify this without necessitating a massive recalculation of the entire NavMesh at runtime, every generated obstacle prefab must contain a NavMeshObstacle component with the **Carve** parameter explicitly enabled.⁵ When the procedural generator instantiates the object, this component automatically calculates the local geometry and cuts a boolean hole into the existing NavMesh graph.⁵

For purely stationary procedural obstacles like trees or fallen pillars, it is a strict technical recommendation to ensure the Carve Only Stationary flag is checked within the inspector. This forces the engine to calculate the pathing void only once upon instantiation, drastically minimizing CPU overhead during active gameplay frames.⁵

User Interface (UI) Design and Implementation

Managing the User Interface in a split-screen configuration demands a specialized Canvas approach. Standard Screen Space - Overlay canvases render UI elements globally across the entire monitor, indiscriminately disregarding camera viewports, which would result in health bars stretching across both players' screens.

To provide truly independent Heads Up Displays (HUDs) containing individual Healthbars and Inventory counts¹, the system utilizes the Screen Space - Camera render mode.

1. Within the Player Prefab, a dedicated UI Canvas is instantiated as a child object.
2. The Canvas Render Mode is set to Screen Space - Camera.
3. The Render Camera slot is dynamically assigned via script to the player's specific split-screen camera upon instantiation.

This guarantees that Player 1's health bar inherently scales and anchors exclusively to the left side of the screen, and Player 2's UI anchors to the right, mitigating visual overlap errors entirely.

Main Menu Architecture

The overarching requirement for UI also specifies the inclusion of a robust Main Menu.¹ The architecture for this involves a separate Scene (e.g., MainMenuScene) containing an Overlay Canvas with the following logic bindings tied to Unity UI Buttons:

- **Start Game Button:** Triggers SceneManager.LoadScene("GameplayScene") to initiate the procedural game loop and hardware polling.
- **Tutorial Button:** Toggles the active state of a UI Panel overlay containing textual

directions delineating the WASD and Gamepad input maps.

- **Credits Button:** Toggles a UI Panel listing all asset attributions and origins. This is a critical requirement for maintaining academic integrity regarding the use of third-party 3D models and audio files.
- **Quit Button:** Executes the Application.Quit() method. For testing purposes, an #if UNITY_EDITOR preprocessor directive utilizing UnityEditor.EditorApplication.isPlaying = false should be included to halt the editor runtime.

Asset Sourcing and Thematic Cohesion

To achieve the mandated "Fantasy Dungeon" aesthetic required for the classes and NPCs, the project must prioritize assets that offer unified visual styles and functional, retargetable rigging parameters. High-fidelity 3D models and animations must share standard humanoid skeletal rigs to allow animation sharing via Unity's Mecanim system.

Recommended Asset Repositories

1. **Mixamo (Adobe):** A primary, freely available resource for humanoid characters and motion-capture data. Mixamo provides pre-rigged models that perfectly align with the core classes²³.
 - *Wizard/Cleric:* Models featuring flowing robes (e.g., the "Kachujin" or "Paladin" models), paired with the "Magic Cast" or "Staff Stance" animation sets.
 - *Fighter:* Heavy armor models (e.g., "Knight D Pele"), utilizing "Great Sword" or "Heavy Shield" locomotion sets.
 - *Assassin:* Lightly armored rogue models, paired with "Dagger Stealth" locomotion and rapid striking animations (e.g., the "Assassin Moves" ActorCore set).²⁴
 - *Hostile NPCs:* Mixamo's "Zombie" and "Mutant" models provide excellent hostile entity assets, packaged with standardized "Zombie Walk" and "Zombie Attack" loops.²⁵
2. **Unity Asset Store:** For environments, static props, and UI elements, stylized low-poly assets drastically reduce rendering overhead while maintaining a cohesive, professional look.
 - *Synty Studios (Polygon Series):* "Polygon Dungeon" or "Polygon Fantasy" packs provide highly modular walls, floors, physical obstacles (chests, barrels, pillars), and consumable visual representations.²⁷
 - *2D UI Assets:* Packs like "Super Retro World" or Kenny Assets provide excellent, scalable UI icons for health bars, speed boosters, and inventory slot backgrounds.²⁹

Comprehensive Implementation Roadmap

To systematically execute this architecture and achieve 100% of the project grade (Basic, Extra, and Bonus parameters), the development lifecycle should proceed through three strict integration phases. Adhering to this sequence minimizes debugging time by ensuring foundational systems are stable before complex logic is layered on top.

Development Phase	Execution Steps	Grade Milestone Addressed

Phase 1: Foundational Framework	1.1: Configure the Unity project. Import the New Input System package. Setup the MainMenuScene with all required UI buttons. 1.2: Construct the CharacterClassSO ScriptableObject system. Build the data assets for the Wizard, Assassin, Fighter, and Cleric. 1.3: Build the decoupled PlayerController and CombatSystem scripts. 1.4: Implement the MultiplayerGameManager. Configure the Control Schemes and write the connection logic to assign appropriate devices to joining players, ensuring the keyboard fallback functions. 1.5: Build the Split-Screen logic. Ensure the instantiated player prefabs correctly assign their internal camera's Viewport Rect.	Hits 50% Basic Requirements
Phase 2: Mechanics and Artificial Intelligence	2.1: Develop the ProceduralGenerator script. Define the map bounds, author the random coordinate loops, and implement the	Hits 50% Extra Requirements (Including the 20% Procedural Map Generation mandate)

	Physics.CheckSphere overlap protection algorithm. 2.2: Bake the base NavMesh. Attach NavMeshObstacle components with Carve enabled to all procedurally spawned physical impediments. 2.3: Build the AI Finite State Machine (EnemyAI). Program the Idle, Patrol, Chase, and Attack sequences. 2.4: Build the Lightweight Inventory. Define the ConsumableSO items. Attach SphereColliders to ground items, intercept them with OnTriggerEnter, and apply their logic.	
Phase 3: Polish and Audio-Visual Integration	3.1: Import Mixamo character models. Map the downloaded animations to Unity Animator Controllers using Blend Trees for smooth locomotion transitions. 3.2: Configure the Screen Space - Camera UI Canvas for each player. Link the health integer to a UI Slider component for real-time visual feedback.	Hits 10% Bonus Requirements

	3.3: Integrate an AudioSource for looping background music. Embed discrete AudioSource components with PlayOneShot logic into the combat and inventory scripts. 3.4: Execute final playtesting utilizing the "thinking-aloud" methodology. Ensure the executable build functions flawlessly outside the editor environment at 1920x1080 resolution.	
--	---	--

Strategic Conclusions and Technical Recommendations

The architecture specified throughout this document provides a highly resilient, scalable blueprint for tackling complex local-multiplayer physics simulations in Unity 2026. By strictly decoupling statistical data from behavioral logic via the implementation of ScriptableObjects, the project completely avoids the technical debt associated with hardcoded conditional statements. This significantly streamlines the integration of the four required classes (Wizard, Assassin, Fighter, Cleric) and their associated consumables, allowing rapid iteration and balance adjustments directly within the editor.

Furthermore, integrating Unity's New Input System early in the pipeline prevents the notorious device-bleeding errors common in legacy local-multiplayer setups. By utilizing explicit PlayerInputManager routing, the system elegantly satisfies the mandatory hardware configuration where Player 1 relies on Keyboard/Mouse, Player 2 utilizes a Gamepad, and a fallback state seamlessly permits both players to share a single keyboard without input conflict. Combining these robust interaction systems with the procedural generation heuristic and dynamic NavMesh carving ensures a stable, infinitely replayable dungeon crawler that unequivocally fulfills, and structurally exceeds, the academic criteria mandated by the Graphics Unity Project 2026 rubric.

Πηγές αναφοράς

1. Graphics_Unity_Project_2026_Split_Screen_Game.pdf
2. Create modular game architecture with ScriptableObjects - Unity, πρόσβαση Μαρτίου 31, 2026,
<https://unity.com/resources/create-modular-game-architecture-scriptableobjects-unity-6>
3. Split Screen: New Input System & Cinemachine - One Wheel Studio, πρόσβαση Μαρτίου 31, 2026,
<https://onewheelstudio.com/blog/2022/2/3/split-screen-multi-player-with-unitys-new-input-system>
4. Spawn and Place NavMeshAgents on a NavMesh at Runtime | AI Series Part 4 | Unity Tutorial - YouTube, πρόσβαση Μαρτίου 31, 2026,
<https://www.youtube.com/watch?v=5uO0dXYbL-s>
5. About NavMesh Obstacles | AI Navigation | 2.0.12 - Unity - Manual, πρόσβαση Μαρτίου 31, 2026,
<https://docs.unity3d.com/Packages/com.unity.ai.navigation@2.0/manual/AboutObstacles.html>
6. Building a modular ability system with Scriptable Objects. Need help. : r/Unity3D - Reddit, πρόσβαση Μαρτίου 31, 2026,
https://www.reddit.com/r/Unity3D/comments/1icgtjb/building_a_modular_ability_system_with_scriptable/
7. Architect your code for efficient changes and debugging with ScriptableObjects | Unity, πρόσβαση Μαρτίου 31, 2026,
<https://unity.com/how-to/architect-game-code-scriptable-objects>
8. Separate Game Data and Logic with ScriptableObjects - Unity, πρόσβαση Μαρτίου 31, 2026,
<https://unity.com/how-to/separate-game-data-logic-scriptable-objects>
9. Create an ability system using Scriptable Objects | by Daniel Ercilio Del Rosario Guerra, πρόσβαση Μαρτίου 31, 2026,
<https://ddelrosarioguerra.medium.com/create-an-ability-system-using-scriptable-objects-6b91d6775cdd>
10. Item/Ability system using scriptableobjects in Unity - Stack Overflow, πρόσβαση Μαρτίου 31, 2026,
<https://stackoverflow.com/questions/51886841/item-ability-system-using-scriptableobjects-in-unity>
11. Class PlayerInput | Input System | 1.1.1 - Unity - Manual, πρόσβαση Μαρτίου 31, 2026,
<https://docs.unity3d.com/Packages/com.unity.inputsystem@1.1/api/UnityEngine.InputSystem.PlayerInput.html>
12. Unity 6 Split-Screen Multiplayer (2-4 Players) with the New Input System - YouTube, πρόσβαση Μαρτίου 31, 2026,
<https://www.youtube.com/watch?v=55MPgeVaqpU>
13. Set up split-screen rendering | Universal RP | 14.0.12 - Unity - Manual, πρόσβαση Μαρτίου 31, 2026,

- <https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal@14.0/manual/rendering-to-the-same-render-target.html>
14. Split-screen manager easy with Viewport rect : r/Unity3D - Reddit, πρόσβαση Μαρτίου 31, 2026, https://www.reddit.com/r/Unity3D/comments/4cc0gc/splitscreen_manager_easy_with_viewport_rect/
 15. How To Create A Split Screen In Unity - YouTube, πρόσβαση Μαρτίου 31, 2026, <https://www.youtube.com/watch?v=rw2VKAdTdgQ>
 16. (Unity) How to effectively manage an inventory system using ScriptableObjects? - Reddit, πρόσβαση Μαρτίου 31, 2026, https://www.reddit.com/r/gamedev/comments/1n0is0v/unity_how_to_effectively_manage_an_inventory/
 17. Pickup Items to Inventory on Trigger Collision | Unity 2018 Tutorial - YouTube, πρόσβαση Μαρτίου 31, 2026, <https://www.youtube.com/watch?v=12AbcoLrYKM>
 18. Procedural Generation Basics: Prevent Overlap Spawns With Boxcasts In C# [Unity Tutorial], πρόσβαση Μαρτίου 31, 2026, <https://www.youtube.com/watch?v=IFmKZskl45I>
 19. Poisson Disk Sampling | Dev.Mag, πρόσβαση Μαρτίου 31, 2026, <http://devmag.org.za/2009/05/03/poisson-disk-sampling/>
 20. Spawning random objects without collisions in Unity [RNDBITS-039] - YouTube, πρόσβαση Μαρτίου 31, 2026, <https://www.youtube.com/watch?v=ENEtzLePZbQ>
 21. Spawn Objects at Random Position in Unity - YouTube, πρόσβαση Μαρτίου 31, 2026, <https://www.youtube.com/watch?v=lbiwNnOv5So>
 22. Creating a NavMesh Obstacle - Unity - Manual, πρόσβαση Μαρτίου 31, 2026, <https://docs.unity3d.com/550/Documentation/Manual/nav-CreateNavMeshObstacle.html>
 23. Mixamo, πρόσβαση Μαρτίου 31, 2026, <https://www.mixamo.com/>
 24. Assassin Moves Animation Pack Review - YouTube, πρόσβαση Μαρτίου 31, 2026, <https://www.youtube.com/watch?v=LNSAEplkFIU>
 25. Animated Wizard Character Model by Gw1p - Itch.io, πρόσβαση Μαρτίου 31, 2026, <https://gw1p.itch.io/animated-wizard-character-model>
 26. 100 FREE Fantasy Game Characters - Asset Pack! - YouTube, πρόσβαση Μαρτίου 31, 2026, <https://www.youtube.com/watch?v=pOC2-2EArgQ>
 27. Unity Input System Demystified: 3 Easy Workflows You Need to Know - YouTube, πρόσβαση Μαρτίου 31, 2026, <https://www.youtube.com/watch?v=z5zShkCR0mg>
 28. Medieval Game Assets | Characters, Monsters & More - Unity Asset Store, πρόσβαση Μαρτίου 31, 2026, <https://assetstore.unity.com/popular-assets/medieval-game-assets>
 29. Procedural Placement of Items and Enemies in Unity 2D - YouTube, πρόσβαση Μαρτίου 31, 2026, <https://www.youtube.com/watch?v=t1a1QBcfRIM>